RoyalPark — AzeriCard Payment Integration: Technical Specification

Version: 2.0
Date: 2026-03-10
Prepared by: RoyalPark Development Team
Audience: AzeriCard Technical Integration Team

---

1. About the System

1.1 What is RoyalPark?

RoyalPark is a **residential property management system** (CRM) used by the management company of a residential complex (villas/houses). It covers:

- **Resident management** — registration of properties, owners, tenants
- **Utility metering** — electricity, gas, water, sewerage readings
- **Monthly invoicing** — automatic invoice generation based on meter readings and fixed tariffs
- **Online payments** — residents pay invoices or top up their advance balance through the web portal
- **Advance balance** — a prepaid wallet that can automatically cover future invoices

1.2 Technical Stack

Component	Technology
Backend	Python 3.13, FastAPI, SQLAlchemy ORM
Database	PostgreSQL
Frontend	HTML/CSS/JavaScript (SPA), served over HTTPS
Authentication	Session-based (cookies), role-based access

1.3 Users

Role	Description
Resident (Sakin)	Property owner or tenant — views invoices, makes online payments
Admin / Operator	Management staff — manages invoices, tariffs, manually records cash/transfer payments

---

2. Three Bank Accounts (Three Terminals Required)

RoyalPark needs **three separate AzeriCard terminals**, each linked to a different bank account. This is a core business requirement because the management company must route payments to different accounts depending on the type of service being paid.

2.1 Terminal Categories

#	Terminal Name	Bank Account Purpose	Services Routed Here
1	**Utility**	Communal services account	Electricity, Gas, Water, Sewerage, Fixed communal tariffs
2	**Maintenance**	Property maintenance account	Service fee, Rent, Construction
3	**Advance**	Prepaid balance account	Advance top-ups (no invoice)

2.2 How the System Determines Which Terminal to Use

The system classifies invoice line items by their service type. The mapping is deterministic:

Service Type	Terminal
Electricity (ELECTRIC)	Utility
Gas (GAS)	Utility
Water (WATER)	Utility
Sewerage (SEWERAGE)	Utility
Service fee (SERVICE)	Maintenance
Rent (RENT)	Maintenance

Service Type	Terminal
Construction (CONSTRUCTION)	Maintenance
Advance top-up (no invoice)	Advance

Important: When a resident pays an invoice, the user interface enforces selection of services from **only one terminal category at a time**. For example, if a resident selects "Electricity" (Utility), all Maintenance items become disabled. This means each payment transaction goes to exactly one terminal — there are never mixed-terminal transactions.

---

3. User Payment Journey — Step by Step

3.1 Scenario A: Paying an Invoice

Below is the complete user journey for paying an invoice via card:

Step 1 — Resident opens their portal

The resident logs into the web portal at `https://royalpark.az` and navigates to "My Invoices" (Hesablarım).

Step 2 — Resident selects an invoice

The system displays all unpaid or partially paid invoices. The resident clicks on an invoice to see its line items:

```
Invoice: INV-37/03/2026-01            Period: March 2026
──────────────────────────────────────────────────────────
☑  Electricity   150 kWh              45.20 AZN  [Utility]
☑  Gas            28 m³               12.80 AZN  [Utility]
☑  Water          15 m³                8.50 AZN  [Utility]
☑  Sewerage                            3.40 AZN  [Utility]
☐  Service fee                        25.00 AZN  [Maintenance]  ← disabled
☐  Rent                              200.00 AZN  [Maintenance]  ← disabled
──────────────────────────────────────────────────────────
Selected: 69.90 AZN  (4 items, all Utility)
```

As shown above, once the resident selects any Utility item, all Maintenance items become disabled (grayed out), and vice versa. This ensures each transaction goes to a single terminal.

Step 3 — Resident proceeds to payment

The resident clicks "Pay" (Ödəmək). The system shows a payment summary page:

```
┌──────────────────────────────────────┐
│  Payment Summary                     │
│                                      │
│  Invoice: INV-37/03/2026-01          │
│  Amount:  69.90 AZN                  │
│  Type:    Utility services           │
│                                      │
│  ┌──────────────┐                    │
│  │ VISA ****7899 │  ← saved card     │
│  └──────────────┘                    │
│                                      │
│  [ + Pay with new card ]             │
│  [ Pay with saved card ]             │
│                                      │
└──────────────────────────────────────┘
```

The resident can either: - **Use a previously saved card** (if available) — selecting it from the list - **Pay with a new card** — by clicking "+ Pay with new card"

Step 4 — Redirect to AzeriCard gateway

The system creates a payment request and redirects the resident's browser to the AzeriCard 3D Secure page. **The resident never enters card details on our site** — all card data (PAN, CVV, OTP) is entered on the AzeriCard-hosted page.

```
Browser redirects to:  https://mpi.3dsecure.az/cgi-bin/cgi_link
Form data:
  AMOUNT    = 69.90
  CURRENCY  = AZN
  ORDER     = RP20260310143012001
  TERMINAL  = E1000001              ← Utility terminal
  TRTYPE    = 0
  MERCH_NAME = RoyalPark
  BACKREF   = https://royalpark.az/api/azericard/callback
  TIMESTAMP = 20260310143012
  NONCE     = a1b2c3d4e5f67890...
  P_SIGN    = <RSA-SHA256 signature>
```

Step 5 — Resident enters card details on AzeriCard page

On AzeriCard's 3D Secure page, the resident enters: - Card number (PAN, 16 digits) - Expiry date (MM/YY) - CVV/CVC code - OTP (one-time password from SMS)

Step 6 — AzeriCard processes the payment and sends callback

After the card is charged, AzeriCard sends a server-to-server callback to our backend:

```
POST https://royalpark.az/api/azericard/callback
```

```
ACTION=0          ← 0 = success
ORDER=RP20260310143012001
AMOUNT=69.90
CURRENCY=AZN
RRN=123456789012
INT_REF=ABCDEF123456
APPROVAL=A12345
RC=00
TERMINAL=E1000001
P_SIGN=<signature>
```

Step 7 — Backend processes the callback

Our backend: 1. Verifies the P_SIGN signature using AzeriCard's public key 2. Confirms ACTION=0 (success) and RC=00 3. Creates a Payment record in the database 4. Applies the payment to the invoice (marks invoice lines as paid) 5. Saves the card token (if returned) for future use

Step 8 — Resident is redirected back

AzeriCard redirects the resident's browser to our success or failure page: - **Success:** `https://royalpark.az/api/azericard/success?ORDER=RP20260310143012001` - **Failure:** `https://royalpark.az/api/azericard/fail?ORDER=RP20260310143012001`

3.2 Scenario B: Advance Balance Top-Up

1. Resident navigates to the payment page and selects "Top up balance" (Avans artır)
2. Resident enters the desired amount (e.g. 100.00 AZN) — there is no invoice involved
3. System uses the **Advance terminal** (Terminal #3)
4. Same redirect flow to AzeriCard gateway
5. After success, the amount is credited to the resident's advance balance
6. This balance is automatically applied to future invoices when they become due

3.3 Scenario C: Paying Maintenance Items

Identical to Scenario A, but the resident selects Maintenance items (Service fee, Rent), and the payment is routed to **Terminal #2 (Maintenance)**.

---

4. Card Tokenization (Saved Cards)

4.1 Business Need

Residents pay monthly recurring bills. To improve the experience, we want to **save the card after the first successful payment** so that future payments require fewer steps (no need to re-enter the full card number every time).

4.2 What We Store

Field	Example	Description
token_id	`"TKN_abc123..."`	Opaque token returned by AzeriCard
masked_pan	`"****7899"`	Last 4 digits only — shown to the user
card_brand	`"VISA"` or `"MC"`	Determined from BIN
expiry_month	`6`	Card expiry month
expiry_year	`2030`	Card expiry year

We do NOT store the full PAN, CVV, or any sensitive card data. The system is entirely out of PCI scope — all card data entry happens on AzeriCard's hosted 3D Secure page.

4.3 Flow — First Payment (Token Acquisition)

```
Resident                    Our Backend                AzeriCard
   |                            |                          |
   |  Pays with new card        |                          |
   |──────────────────────────►|                          |
   |                            |  Initiate with           |
   |                            |  save_card flag          |
   |                            |──────────────────────────►|
   |                            |                          |
   |  Enters card on            |                          |
   |  AzeriCard page ──────────────────────────────────────►|
   |                            |                          |
   |                            |  Callback includes       |
   |                            |  token + masked PAN      |
   |                            |◄──────────────────────────|
   |                            |                          |
   |                            |  Saves token to DB       |
   |                            |                          |
   |  ◄──────── Success page    |                          |
```

4.4 Flow — Subsequent Payment (Using Saved Card)

```
Resident                    Our Backend                AzeriCard
   |                            |                          |
```

```
    │  Selects "VISA ****7899"   │                          │
    │  and clicks Pay            │                          │
    ├────────────────────────────►│                          │
    │                            │  Sends payment with      │
    │                            │  token (no card form)    │
    │                            ├──────────────────────────►│
    │                            │                          │
    │                            │  Result                  │
    │                            │◄──────────────────────────┤
    │                            │                          │
    │  ◄──────── Success/Fail    │                          │
```

4.5 Card Management

Residents can: - **View** their saved cards (displayed as `VISA ****7899`) - **Delete** a saved card (removes the token from our database) - **Set a default** card for quick one-click payments - A maximum of **5 saved cards** per user is enforced

---

5. Payment Restrictions and Business Rules

Rule	Description
Single category per transaction	A resident cannot mix Utility and Maintenance items in one payment. The UI enforces this.
No overpayment of invoices	When paying an invoice, the amount cannot exceed the remaining unpaid balance of that invoice.
Advance has no upper limit	When topping up the advance balance, the resident can enter any amount.
Automatic advance application	When an invoice's due date arrives and the resident has advance balance, the system automatically deducts from the advance to cover the invoice.
Partial payment allowed	A resident can pay less than the full invoice amount (e.g., pay only some line items).

---

6. Technical Details

6.1 API Endpoints

Endpoint	Method	Description
`/api/azericard/initiate`	POST	Creates a payment transaction and returns gateway parameters
`/api/azericard/callback`	POST	Receives server-to-server callback from AzeriCard after 3DS
`/api/azericard/status/{order_id}`	GET	Queries the current status of a transaction (TRTYPE 90)
`/api/azericard/complete`	POST	Completes a pre-authorized transaction (TRTYPE 21)
`/api/azericard/success`	GET	Success redirect page shown to the resident
`/api/azericard/fail`	GET	Failure redirect page shown to the resident
`/api/azericard/saved-cards`	GET	Lists the resident's saved cards
`/api/azericard/saved-cards/{id}`	DELETE	Removes a saved card

6.2 Initiate Request / Response

Request:

```
{
    "resident_id": 42,
    "amount": 69.90,
    "invoice_id": 1001,
    "description": "Payment for March 2026",
    "terminal_category": "utility",
    "saved_card_id": null
}
```

Response:

```
{
    "ok": true,
    "order_id": "RP20260310143012001",
    "amount": "69.90",
    "gateway_url": "https://mpi.3dsecure.az/cgi-bin/cgi_link",
    "method": "POST",
    "terminal_category": "utility",
    "params": {
        "AMOUNT": "69.90",
        "CURRENCY": "AZN",
        "ORDER": "RP20260310143012001",
        "TERMINAL": "E1000001",
        "TRTYPE": "0",
```

```
        "MERCH_NAME": "RoyalPark",
        "TIMESTAMP": "20260310143012",
        "NONCE": "a1b2c3d4e5f67890...",
        "BACKREF": "https://royalpark.az/api/azericard/callback",
        "P_SIGN": "<RSA-SHA256 signature>"
    }
}
```

The frontend takes this response, creates a hidden HTML form with all the `params`, and submits it to `gateway_url`. This redirects the browser to AzeriCard's 3D Secure page.

6.3 Callback Data (from AzeriCard)

```
POST /api/azericard/callback

ACTION=0
ORDER=RP20260310143012001
AMOUNT=69.90
CURRENCY=AZN
RRN=123456789012
INT_REF=ABCDEF123456
APPROVAL=A12345
RC=00
TERMINAL=E1000001
TRTYPE=0
TIMESTAMP=20260310143500
NONCE=f0e1d2c3b4a59876
P_SIGN=<signature>
```

6.4 Transaction Types (TRTYPE)

TRTYPE	Operation	When Used
0	Purchase	Standard payment
21	Completion	Pre-authorization completion
22	Reversal	Refund / cancel
90	Status Query	Check transaction status by ORDER

6.5 P_SIGN Digital Signature

Every request is signed with **RSA-2048 SHA-256**:

- **Outgoing (to AzeriCard):** We sign the concatenation of: `AMOUNT;CURRENCY;TERMINAL;TRTYPE;TIMESTAMP;NONCE;BACKREF` using the **merchant's private key**
- **Incoming (from AzeriCard):** We verify using **AzeriCard's public key** against: `ACTION;AMOUNT;CURRENCY;ORDER;RRN;INT_REF;TRTYPE;TERMINAL;TIMESTAMP;NONCE`

6.6 Security

- **No PCI scope:** Our system never receives, processes, or stores full card numbers, CVV, or expiry dates
- Card details are entered **only** on the AzeriCard 3D Secure hosted page
- We store only: **token** (from AzeriCard) and **masked PAN** (last 4 digits)
- All communication occurs over **HTTPS/TLS**
- Duplicate callbacks are safely ignored (idempotent processing)

---

7. Database Schema (Payment-Related)

```
online_transactions
├── id                  (PK)
├── payment_id          (FK → payments, nullable)
├── resident_id         (FK → residents)
├── invoice_id          (FK → invoices, nullable — null for advance top-ups)
├── order_id            (VARCHAR, UNIQUE — e.g. "RP20260310143012001")
├── amount_total        (NUMERIC 12,2)
├── currency            (VARCHAR — "AZN")
├── trtype              (VARCHAR — "0")
├── terminal_category   (VARCHAR — "utility" | "maintenance" | "advance")
├── rrn                 (VARCHAR — from AzeriCard callback)
├── int_ref             (VARCHAR — from AzeriCard callback)
├── approval            (VARCHAR — from AzeriCard callback)
├── action_code         (VARCHAR — "0" = success)
├── rc                  (VARCHAR — "00" = success)
├── gateway_status      (VARCHAR — INITIATED → CONFIRMED / DECLINED / ...)
├── request_payload     (JSON — full request sent to gateway)
├── callback_payload    (JSON — full callback received)
├── created_at          (TIMESTAMP)
└── updated_at          (TIMESTAMP)

saved_cards
├── id                  (PK)
├── user_id             (FK → users)
├── token_id            (VARCHAR — opaque AzeriCard token)
├── masked_pan          (VARCHAR — "****7899")
├── card_brand          (VARCHAR — "VISA" | "MC")
```

```
├── expiry_month        (INTEGER)
├── expiry_year         (INTEGER)
├── is_default          (BOOLEAN)
└── created_at          (TIMESTAMP)
```

---

8. Environment & URLs

Parameter	Test Value	Production Value
Gateway URL	`https://testmpi.3dsecure.az/cgi-bin/cgi_link`	*(provided by AzeriCard)*
Callback URL (BACKREF)	`https://<domain>/api/azericard/callback`	Same pattern
Success redirect	`https://<domain>/api/azericard/success`	Same pattern
Fail redirect	`https://<domain>/api/azericard/fail`	Same pattern
Currency	AZN	AZN
Language	az / en / ru	az

---

9. Questions for AzeriCard

We need AzeriCard's guidance on the following to proceed:

9.1 Three Terminals

1. Can the three terminals share a **single callback URL** (`BACKREF`), or does each terminal require a separate endpoint?
2. Will each terminal have its **own RSA key pair**, or is one merchant key pair shared across all terminals?
3. Can we proceed with testing on one terminal first and add the other two later?

9.2 Card Tokenization

1. What fields does the callback return for **card-on-file tokens**? What is the parameter name and format?
2. How do we initiate a **token-based payment** (without redirecting to the 3D Secure page)? Is there a separate TRTYPE or an additional field in the request?

3. Does a saved token work across all three terminals, or is it terminal-specific?

9.3 General

1. Is there support for **recurring/scheduled** payments using saved tokens?
2. What are the **test terminal IDs** and keys for the three accounts?
3. Is there a sandbox endpoint for testing tokenization?

Appendix A: Test Environment

Test cards:

PAN	Expiry	CVV	OTP
4127208108785956	06/30	595	1111
5522099313278830	06/30	669	1111

Sandbox tools: - P_SIGN Calculator: `https://testsite.3dsecure.az/sandbox/p-sign.php` - Payment Auth Tester: `https://testsite.3dsecure.az/sandbox/auth.php` - Transaction Status: `https://testsite.3dsecure.az/sandbox/transaction_status.php` - Checkout/Reversal: `https://testsite.3dsecure.az/sandbox/checkout-reversal.php`

Appendix B: Visual Flow Summary

```
┌──────────────────────────────────────────────────────────────────┐
│                      RESIDENT WEB PORTAL                         │
│                                                                  │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────────────┐   │
│  │ My       │    │ Select       │    │ Payment page:        │   │
│  │ Invoices │───►│ line items   │───►│ - Choose saved card  │   │
│  │          │    │ (one         │    │   OR new card        │   │
│  │          │    │  category    │    │ - Confirm amount     │   │
│  └──────────┘    │  only!)      │    └──────────┬───────────┘   │
│                  └──────────────┘               │               │
│                                                 │               │
│                                                 │               │
│  ┌──────────┐                                   │               │
│  │ Top up   │───────────────────────────────────┤               │
│  │ Advance  │  (no invoice, any amount)         │               │
│  └──────────┘                                   │               │
```

```
└──────────────────────────────────────────┬──────────────────────┘
                                           │
                                           ▼
                    ┌──────────────────────────────────────┐
                    │         OUR BACKEND (FastAPI)        │
                    │                                      │
                    │  1. Determine terminal category      │
                    │  2. Generate ORDER, sign P_SIGN      │
                    │  3. Create DB record                 │
                    │  4. Return gateway params             │
                    └──────────────┬───────────────────────┘
                                   │
                    ┌──────────────┼───────────────────────┐
                    │              ▼                       │
                    │   REDIRECT TO AZERICARD 3D SECURE    │
                    │                                      │
                    │   Terminal E1000001 (Utility)        │
                    │   Terminal E2000002 (Maintenance)    │
                    │   Terminal E3000003 (Advance)        │
                    │                                      │
                    │   Resident enters: PAN, CVV, OTP     │
                    │                                      │
                    └──────────────┬───────────────────────┘
                                   │
                                   ▼
                    ┌──────────────────────────────────────┐
                    │     CALLBACK TO OUR BACKEND          │
                    │                                      │
                    │  1. Verify P_SIGN                    │
                    │  2. Check ACTION=0, RC=00            │
                    │  3. Record payment                   │
                    │  4. Apply to invoice / add to advance │
                    │  5. Save card token (if returned)    │
                    │  6. Redirect resident to result page  │
                    └──────────────────────────────────────┘
```